

A third choice, and perhaps the best, is to create your own custom exception. This is easy to do, but involves a technique called inheritance (which we will discuss at length in Chapter 10). Here is all you need to create your own exception:

```
# Define a custom exception
class <CustomExceptionName>(Exception):
    pass
```

You supply a unique name for your exception. You can then `raise` your custom exception in your code. Creating your own exceptions means you can explicitly check for these exceptions by name in a higher level of your code. In the next section, we'll rewrite the code of our bank example so that we raise a custom exception in our `Bank` and `Account` classes and check for and report the error in the main code. The main code will report the error but allow the program to continue running.

In the typical case, the `raise` statement causes the current function or method to exit and transfers control back to the caller. If the caller contains an `except` clause that catches the exception, execution continues inside that `except` clause. Otherwise, that function or method exits. This process is repeated until an `except` clause catches the exception. Control is transferred back through the sequence of calls, and if no `except` clause catches the exception, the program quits and Python displays the error.

Using Exceptions in Our Bank Program

We can now rewrite all three levels of our program (main, `Bank`, and `Account`) to signal errors with `raise` statements and to handle errors using `try/except` blocks.

Account Class with Exceptions

[Listing 4-9](#) is a new version of the `Account` class rewritten to use exceptions and optimized so that no code is repeated. We start by defining a custom `AbortTransaction` exception, which will be raised if we discover some error while a user is attempting to do a transaction in our bank.